

WEB TECHNOLOGY

Academic Learning Module

GeeksforGeeks Content Extractor

Subject Category	Web Technology
Total Topics	12
Date Generated	March 17, 2026
Source	www.geeksforgeeks.org
Prepared by	Web Technology Student

Table of Contents

01. HTML Tutorial	Easy
02. CSS Tutorial — Cascading Style Sheets	Easy
03. JavaScript — Complete Tutorial	Medium
04. HTTP — HyperText Transfer Protocol	Easy
05. Domain Name System (DNS)	Medium
06. REST API — Introduction and Design Principles	Medium
07. Web Cookies — Storage and Security	Easy
08. TCP/IP Model — Layers and Protocols	Medium
09. Responsive Web Design (RWD)	Easy
10. JSON — JavaScript Object Notation	Easy
11. AJAX — Asynchronous JavaScript and XML	Medium
12. WebSocket — Full-Duplex Real-Time Communication	Hard

01 HTML Tutorial

Easy

KEY TECHNICAL CONCEPTS

HTML (HyperText Markup Language) is the standard markup language used to create web pages. HTML describes the structure of a web page using a series of elements. An HTML element is defined by a start tag, some content, and an end tag. Browsers do not display HTML tags, but use them to render the content of the page. HTML5 is the latest version and supports multimedia, semantic elements like , , , and native form validation.

CODE IMPLEMENTATION

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0"> <title>My First HTML Page</title> </head> <body>
<h1>Hello, World!</h1> <p>This is a paragraph.</p> <a href="https://www.geeksforgeeks.org">Visit
GFG</a> </body> </html>
```

COMPLEXITY ANALYSIS

Not Applicable — HTML is a markup language, not an algorithm. Rendering complexity depends on DOM size: $O(n)$ for parsing, where n = number of nodes.

REFERENCES / RELATED LINKS

- [HTML5 Specification](#)
- [HTML Forms](#)
- [Semantic HTML](#)

02 CSS Tutorial — Cascading Style Sheets

Easy

KEY TECHNICAL CONCEPTS

CSS (Cascading Style Sheets) is used to style and layout web pages. It controls how HTML elements appear on screen: colors, fonts, spacing, positioning, and animations. CSS follows a cascade model where specificity and source order determine which rule wins. The box model (content, padding, border, margin) is central to layout. CSS3 introduced flexbox, grid, custom properties (variables), and keyframe animations enabling complex, responsive layouts without JavaScript.

CODE IMPLEMENTATION

```
/* Basic CSS Selectors and Box Model */ body { font-family: 'Segoe UI', sans-serif; margin: 0; background: #f5f5f5; } .container { display: flex; gap: 1rem; padding: 20px; } .card { background: white; border-radius: 8px; box-shadow: 0 2px 8px rgba(0,0,0,0.1); padding: 1rem; flex: 1; }
```

COMPLEXITY ANALYSIS

Not Applicable — CSS rendering complexity is $O(n * m)$ in worst case where n = elements and m = matching rules. Modern browsers use optimized selector matching.

REFERENCES / RELATED LINKS

- [CSS Flexbox Guide](#)
- [CSS Grid Layout](#)
- [CSS Animations](#)

03 JavaScript — Complete Tutorial

Medium

KEY TECHNICAL CONCEPTS

JavaScript is a lightweight, interpreted, object-oriented language with first-class functions. It is primarily used for client-side scripting but also powers server-side development via Node.js. JavaScript is single-threaded with an event loop enabling asynchronous programming through callbacks, Promises, and `async/await`. ES6+ introduced arrow functions, destructuring, spread operators, modules, classes, and the `let/const` block scoping. The language is dynamically typed and prototype-based.

CODE IMPLEMENTATION

```
// Async/Await with Fetch API
async function fetchData(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Fetch failed:', error);
    throw error;
  }
}

// Arrow function & destructuring
const greet = ({ name, age }) => `Hello ${name}, age ${age}`;
console.log(greet({ name: 'Alice', age: 25 }));
```

COMPLEXITY ANALYSIS

`Array.sort()`: $O(n \log n)$ average. `Map/Filter/Reduce`: $O(n)$. Object property lookup: $O(1)$ average. Closures add $O(1)$ per captured variable to memory. Promise chaining: $O(n)$ for n chained `.then()` calls.

REFERENCES / RELATED LINKS

- [JavaScript ES6 Features](#)
- [Event Loop Explained](#)
- [Promises and `async/await`](#)

04 HTTP — HyperText Transfer Protocol

Easy

KEY TECHNICAL CONCEPTS

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the World Wide Web. It is a stateless, request-response protocol operating at the application layer of the TCP/IP model. HTTP/1.1 introduced persistent connections; HTTP/2 added multiplexing and header compression; HTTP/3 uses QUIC over UDP for lower latency. Key methods include GET (retrieve), POST (submit data), PUT (update), DELETE, PATCH, and OPTIONS. Status codes: 2xx success, 3xx redirect, 4xx client error, 5xx server error.

CODE IMPLEMENTATION

```
# Python — Making HTTP requests import requests # GET request response = requests.get('https://api.example.com/users', headers={'Authorization': 'Bearer token123'}) print(response.status_code) # 200 data = response.json() # POST request payload = {'username': 'alice', 'password': 'secret'} r = requests.post('https://api.example.com/login', json=payload) print(r.json())
```

COMPLEXITY ANALYSIS

HTTP request round-trip latency: $O(RTT)$. HTTP/2 multiplexing reduces $O(n * RTT)$ to $O(RTT)$ for n parallel requests. Header compression (HPACK) reduces overhead by ~30–85% compared to HTTP/1.1.

REFERENCES / RELATED LINKS

- [HTTP/2 vs HTTP/1.1](#)
- [HTTPS and SSL/TLS](#)
- [HTTP Status Codes](#)

05 Domain Name System (DNS)

Medium

KEY TECHNICAL CONCEPTS

DNS (Domain Name System) is a hierarchical, decentralized naming system for computers, services, or resources connected to the Internet. It translates human-readable domain names (e.g., google.com) into IP addresses (e.g., 142.250.80.46). DNS uses a tree structure: Root zone > Top-Level Domains (TLDs like .com, .org) > Second-Level Domains > Subdomains. The resolution process involves Recursive resolvers, Root nameservers, TLD nameservers, and Authoritative nameservers. DNS records include A, AAAA, CNAME, MX, TXT, NS, and PTR records.

CODE IMPLEMENTATION

```
# Python DNS lookup using socket and dnspython
import socket
import dns.resolver

# Simple hostname resolution
ip = socket.gethostbyname('www.geeksforgeeks.org')
print(f'IP Address: {ip}')

# Query specific DNS record types
rdata in dns.resolver.resolve('geeksforgeeks.org', 'MX'):
print(f'MX: {rdata.exchange} priority {rdata.preference}')
```

COMPLEXITY ANALYSIS

DNS lookup time: $O(1)$ when cached (TTL-based), $O(\log n)$ hierarchical resolution where n = depth of DNS tree (typically ≤ 4 hops). DNS caching at client, resolver, and ISP levels dramatically reduces resolution time from ~100ms to <1ms for cached records.

REFERENCES / RELATED LINKS

- [DNS Record Types](#)
- [DNS Security \(DNSSEC\)](#)
- [CDN and DNS](#)

06 REST API — Introduction and Design Principles

Medium

KEY TECHNICAL CONCEPTS

REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful APIs use HTTP methods to perform CRUD operations on resources identified by URIs. The six REST constraints are: Client-Server separation, Statelessness, Cacheability, Uniform Interface, Layered System, and Code-On-Demand (optional). Resources are represented in JSON or XML. A RESTful API is stateless — each request from the client must contain all the information needed to understand the request.

CODE IMPLEMENTATION

```
# Flask REST API example from flask
import Flask, jsonify, request
app = Flask(__name__)
users = {1: {'name': 'Alice', 'email': 'alice@example.com'}}
@app.route('/api/users', methods=['GET'])
def get_users():
    return jsonify(list(users.values()))
@app.route('/api/users/<int:uid>', methods=['PUT'])
def update_user(uid):
    if uid not in users:
        return jsonify({'error': 'Not found'}), 404
    users[uid].update(request.json)
    return jsonify(users[uid])
if __name__ == '__main__':
    app.run(debug=True)
```

COMPLEXITY ANALYSIS

REST API endpoint complexity depends on implementation. Simple CRUD endpoints: $O(1)$ for in-memory, $O(\log n)$ for indexed DB queries, $O(n)$ for full table scans. Pagination reduces response payload: $O(\text{page_size})$ per request. Caching (ETag, Cache-Control) can reduce server load to $O(1)$ for cached responses.

REFERENCES / RELATED LINKS

- [REST vs SOAP](#)
- [HTTP Methods in REST](#)
- [API Authentication](#)

07 Web Cookies — Storage and Security

Easy

KEY TECHNICAL CONCEPTS

Web cookies are small text files stored on a user's device by the browser when visiting a website. Cookies carry state information (since HTTP is stateless) such as session IDs, user preferences, and authentication tokens. Key attributes: Name, Value, Domain, Path, Expires/Max-Age, Secure (HTTPS only), HttpOnly (no JS access), SameSite (CSRF protection). Types: Session cookies (deleted on browser close), Persistent cookies (survive sessions), Third-party cookies (set by external domains, used for tracking). GDPR requires consent.

CODE IMPLEMENTATION

```
// JavaScript — Reading and Writing Cookies // Set a cookie (expires in 7 days) function
setCookie(name, value, days) { const expires = new Date(); expires.setTime(expires.getTime() +
days * 24 * 60 * 60 * 1000); document.cookie =
`${name}=${value};expires=${expires.toUTCString()};path=/;SameSite=Strict`; } // Get a cookie by
name function getCookie(name) { const cookies = document.cookie.split('; '); const found =
cookies.find(c => c.startsWith(name + '=')); return found ? found.split('=')[1] : null; }
setCookie('theme', 'dark', 30); console.log(getCookie('theme')); // 'dark'
```

COMPLEXITY ANALYSIS

Cookie read/write: $O(n)$ where n = number of cookies in the header string. Browser limit: 4KB per cookie, ~50 cookies per domain. Parsing document.cookie: $O(n)$ string scan. HttpOnly + Secure flags add zero computational overhead but prevent XSS/MITM attacks.

REFERENCES / RELATED LINKS

- [Session vs Cookies](#)
- [CSRF Attack Prevention](#)
- [LocalStorage vs Cookies](#)

08 TCP/IP Model — Layers and Protocols

Medium

KEY TECHNICAL CONCEPTS

The TCP/IP model is a concise framework for the Internet's communication protocols, consisting of four layers: Application (HTTP, FTP, DNS, SMTP), Transport (TCP, UDP), Internet (IP, ICMP, ARP), and Network Access (Ethernet, Wi-Fi). TCP provides reliable, ordered, error-checked delivery via a three-way handshake (SYN, SYN-ACK, ACK). UDP is connectionless and faster but unreliable, suitable for streaming and DNS. IP addressing uses IPv4 (32-bit) or IPv6 (128-bit). Subnetting divides networks using CIDR notation (e.g., 192.168.1.0/24).

CODE IMPLEMENTATION

```
# Python TCP Socket Programming
import socket

# TCP Server
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 9090))
server.listen(5)
print('Server listening on port 9090...')
conn, addr = server.accept()
print(f'Connection from {addr}')
data = conn.recv(1024).decode()
conn.send(f'Echo: {data}'.encode())
conn.close()

# TCP Client
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 9090))
client.send(b'Hello Server!')
response = client.recv(1024)
print(response.decode())
```

COMPLEXITY ANALYSIS

TCP handshake: 1.5 RTT (Round Trip Time) latency overhead. TCP slow start: $O(\log n)$ rounds to reach full bandwidth where n = bandwidth-delay product. UDP: $O(1)$ send latency (no connection setup). IP routing table lookup: $O(\log n)$ with CIDR tries, $O(1)$ with hardware-assisted forwarding.

REFERENCES / RELATED LINKS

- [OSI vs TCP/IP Model](#)
- [TCP vs UDP](#)
- [IPv4 vs IPv6](#)

09 Responsive Web Design (RWD)

Easy

KEY TECHNICAL CONCEPTS

Responsive Web Design (RWD) is an approach to web development that makes web pages render well across a variety of devices and screen sizes. Core principles: fluid grids (percentage widths instead of fixed pixels), flexible images (max-width: 100%), and CSS media queries (@media breakpoints). Mobile-first design means writing CSS for mobile first, then adding complexity for larger screens. Common breakpoints: 576px (sm), 768px (md), 992px (lg), 1200px (xl). Frameworks like Bootstrap and Tailwind CSS implement RWD grid systems.

CODE IMPLEMENTATION

```
/* Responsive Grid with CSS Grid + Media Queries */ .grid { display: grid; grid-template-columns: 1fr; /* Mobile: 1 column */ gap: 1rem; padding: 1rem; } @media (min-width: 768px) { .grid { grid-template-columns: repeat(2, 1fr); /* Tablet: 2 columns */ } } @media (min-width: 1024px) { .grid { grid-template-columns: repeat(3, 1fr); /* Desktop: 3 columns */ } } img { max-width: 100%; height: auto; } /* Flexible images */
```

COMPLEXITY ANALYSIS

Not Applicable — RWD is a design methodology. CSS media query matching is $O(n)$ where n = number of rules. CSS Grid layout computation is $O(n)$ per axis.

REFERENCES / RELATED LINKS

- [CSS Media Queries](#)
- [Bootstrap Grid System](#)
- [Viewport Meta Tag](#)

10 JSON — JavaScript Object Notation

Easy

KEY TECHNICAL CONCEPTS

JSON (JavaScript Object Notation) is a lightweight, text-based, language-independent data interchange format. It is human-readable and easy for machines to parse. JSON data types: string, number, boolean, null, array, and object. JSON is widely used for REST API responses, configuration files, and data storage (NoSQL databases like MongoDB). JSON Schema validates JSON structure. JSON5 extends JSON with comments and trailing commas. JSON Web Tokens (JWT) use Base64-encoded JSON for authentication claims.

CODE IMPLEMENTATION

```
// JSON Parsing and Serialization in JavaScript
const user = { id: 1, name: 'Alice', roles: ['admin', 'user'], address: { city: 'Manila', country: 'PH' }, active: true };
// Serialize to JSON string
const jsonStr = JSON.stringify(user, null, 2);
console.log(jsonStr);
// Parse JSON string back to object
const parsed = JSON.parse(jsonStr);
console.log(parsed.address.city);
// 'Manila'
// Deep clone via JSON (loses functions/undefined)
const clone = JSON.parse(JSON.stringify(user));
```

COMPLEXITY ANALYSIS

JSON.parse(): $O(n)$ where n = string length. JSON.stringify(): $O(n)$ where n = number of key-value pairs recursively. Deeply nested structures may cause $O(d * n)$ where d = depth — use streaming parsers (e.g., jsonstream) for large payloads to avoid memory issues.

REFERENCES / RELATED LINKS

- [JSON vs XML](#)
- [JWT Authentication](#)
- [JSON Schema Validation](#)

11 AJAX — Asynchronous JavaScript and XML

Medium

KEY TECHNICAL CONCEPTS

AJAX (Asynchronous JavaScript and XML) is a web development technique enabling web pages to update asynchronously by exchanging data with a server behind the scenes without reloading the entire page. AJAX uses the XMLHttpRequest (XHR) object or the modern Fetch API. Despite the name, JSON is now the dominant data format over XML. AJAX powers Single Page Applications (SPAs), live search, infinite scroll, and real-time form validation. CORS (Cross-Origin Resource Sharing) headers control cross-domain AJAX requests.

CODE IMPLEMENTATION

```
// Modern AJAX using Fetch API with error handling async function searchTopics(query) { const url = `/api/search?q=${encodeURIComponent(query)}`; // Show loading state document.getElementById('results').innerHTML = '<p>Loading...</p>'; try { const response = await fetch(url, { method: 'GET', headers: { 'Accept': 'application/json' } }); if (!response.ok) throw new Error(`Error: ${response.status}`); const data = await response.json(); renderResults(data.topics); } catch (err) { document.getElementById('results').innerHTML = '<p class='error'>${err.message}</p>`; } }
```

COMPLEXITY ANALYSIS

AJAX request latency: $O(\text{RTT} + \text{server processing time})$. Debounced search: limits to $O(1)$ request per debounce window. Polling: $O(n)$ requests over time n — prefer WebSockets or SSE for real-time updates. JSON parsing on response: $O(n)$ where n = response size.

REFERENCES / RELATED LINKS

- [Fetch API Guide](#)
- [CORS Explained](#)
- [XMLHttpRequest vs Fetch](#)

12 WebSocket — Full-Duplex Real-Time Communication

Hard

KEY TECHNICAL CONCEPTS

WebSocket is a communication protocol providing full-duplex, bidirectional communication channels over a single TCP connection. Unlike HTTP's request-response model, WebSocket allows servers to push data to clients without polling. The handshake begins as an HTTP upgrade request (Upgrade: websocket header). Once established, either side can send messages at any time with low latency. Use cases: live chat, multiplayer games, real-time dashboards, collaborative editing, financial tickers, and IoT sensor streams. Socket.IO is a popular library abstracting WebSocket with fallbacks.

CODE IMPLEMENTATION

```
// WebSocket Client (Browser JavaScript)
const socket = new
WebSocket('wss://chat.example.com/ws');
socket.addEventListener('open', () => {
  console.log('Connected to WebSocket server');
  socket.send(JSON.stringify({ type: 'join', room:
    'general' }));
});
socket.addEventListener('message', (event) => {
  const msg =
    JSON.parse(event.data);
  console.log('Received:', msg);
  appendMessage(msg);
});
socket.addEventListener('close', (event) => {
  console.log(`Disconnected: code=${event.code}`);
  // Implement exponential backoff reconnect
  setTimeout(reconnect, 3000);
});
```

COMPLEXITY ANALYSIS

WebSocket message latency: $O(1)$ per message after connection (no handshake overhead). HTTP polling: $O(n * RTT)$ for n polls vs WebSocket $O(RTT)$ for connection + $O(1)$ per message. Memory: $O(c)$ where c = number of active connections (each ~few KB on server). At scale, 100K concurrent WebSocket connections $\approx$ 1-5GB server RAM depending on payload.

REFERENCES / RELATED LINKS

- [WebSocket vs HTTP](#)
- [Socket.IO Introduction](#)
- [Server-Sent Events \(SSE\)](#)